

ENVS-193DS_Infographic_Visualizations

Ella Menin

2026-05-28

Table of contents

Setting up	1
Creating visualizations!	2
Visualization 1: Jitter plot – relationship between water intake and temperature that day	2
Visualization 2: Bubble plot – daily steps, water intake, shift duration, and illness .	5
Visualization 3: Ridgeline plot – showing how much water I drink when I have a sinus infection vs do not have a sinus infection	8
Bonus: Visualization 4: Pie chart – showing how many days I had a sinus infection for fun	10
Write-up: Part 4	13
General information about design and visualization	13
Sources and processes	14
Inspirations	14
Data creation process	14
Tools utilized	15

Setting up

```
# reading in package: ggribges
library(ggribges)
# reading in package: dplyr
library(dplyr)
# reading in package: janitor
library(janitor)
```

```
# reading in package: ggplot2
library(ggplot2)
# reading in package: fontawesome
library(fontawesome)
# reading in package: showtext
library(showtext)
```

```
font_add_google("Poppins", "poppins")
showtext_auto()
```

```
# creating object personal_data from reading in personal data file
personal_data <- read.csv("data/personal_data.csv")
```

```
# creating new object my_data_clean from personal_data data frame
my_data_clean <- personal_data |>
  # using janitor to clean names (underscores and all lowercase)
  janitor::clean_names() |>
  # using filter removing certain rows
  filter(!is.na(temperature_h_w_c), # removes rows where temperature is n/a
         temperature_h_w_c != "") |> # removes rows where temperature is an
  ↪ empty string
  # mutating temperature variable to change levels, influences the order in
  ↪ which they will appear in a plot
  mutate(temperature_h_w_c = factor(temperature_h_w_c,
                                     # levels go hot -> warm -> cold
                                     levels = c("H", "W", "C"),
                                     # changing from letters to whole words
                                     ↪ (easier to understand in plot)
                                     labels = c("Hot", "Warm", "Cold"))))
```

Creating visualizations!

Visualization 1: Jitter plot – relationship between water intake and temperature that day

```
# base layer ggplot with my_data_clean data frame
ggplot(data = my_data_clean,
```

```

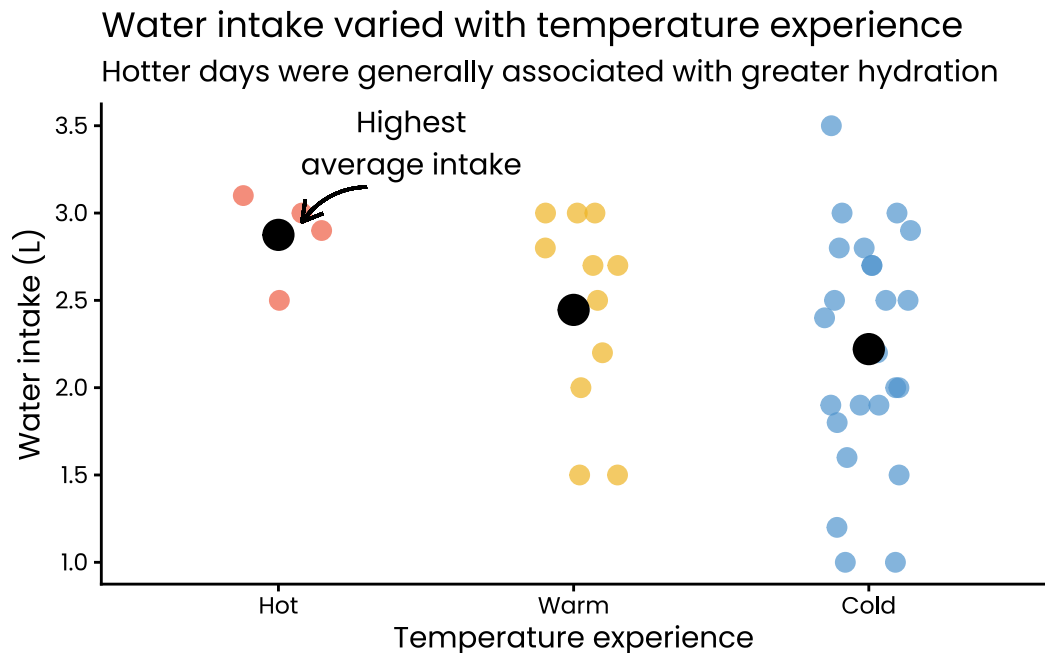
    # x axis is temperature (hot, warm, or cold)
    mapping = aes(x = temperature_h_w_c,
        # y axis is water intake in liters
        y = water_intake_l,
        # color by temperature
        color = temperature_h_w_c)) +
# first layer jitter to show individual data points
geom_jitter(width = 0.15, # controlling horizontal spread
    height = 0, # controlling vertical spread
    size = 3, # making the dots easy to see
    alpha = 0.7) + # slight transparency of dots
# adding in the mean point to make it easier to read
stat_summary(fun = mean,
    # point to represent mean
    geom = "point",
    # mean point size bigger than individual point size
    size = 5,
    # making mean points black so they can easily be distinguished
    color = "black") +
# manually setting colors for temperature
scale_color_manual(values = c(
    # hot is a red color because duh
    "Hot" = "tomato2",
    # warm is a golden color because it is warm but not too... hot
    "Warm" = "goldenrod2",
    # cold is a blue color because duh
    "Cold" = "steelblue3"
)) +
# setting title, subtitle, and axes titles
labs(
    # title explains what is happening in figure
    title = "Water intake varied with temperature experience",
    # subtitle provides a conclusion from the figure
    subtitle = "Hotter days were generally associated with greater hydration",
    # setting x axis title: temperature experience
    x = "Temperature experience",
    # setting y axis title: water intake
    y = "Water intake (L)"
) +
    # adding annotation to point out that hot days had highest average intake
annotate(
    "text",
    # placement: text 1.45 along x axis

```

```

x = 1.45,
# placement: text 3.4 along y axis
y = 3.4,
# labeling arrow with sentence break for aesthetics
label = "Highest\naverage intake",
# cute consistent font
family = "poppins"
) +
# adding the arrow pointing to highest average intake
geom_curve(
  # placement (where curve starts): arrow 1.3 along x axis
  x = 1.3,
  # placement (where curve starts): arrow 3.15 along y axis
  y = 3.15,
  # placement (where curve ends): arrow 1.08 along x axis
  xend = 1.08,
  # placement (where curve ends): arrow 2.95 along y axis
  yend = 2.95,
  # curvature is 0.25
  curvature = 0.25,
  # creating arrow and how big it is
  arrow = arrow(length = unit(0.3, "cm")),
  # setting color back for readability
  color = "black",
  # line width of arrow
  linewidth = 0.6
) +
# cute theme
theme_classic() +
# adding the cutest custom font ever from showtext
theme(
  text = element_text(family = "poppins")) +
# removing legend because i do not find it to be necessary here
theme(legend.position = "none")

```



```
# code to save jitterplot as a png with specified dimensions
ggsave("visualizations/jitterplot.png", width = 8, height = 6, dpi = 300)
```

Visualization 2: Bubble plot – daily steps, water intake, shift duration, and illness

```
# creating new object bubble data out of my_data_clean data frame for bubble
  ↳ plot
bubble_data <- my_data_clean |>
  # using filter function to get tell r to ignore n/a values for following
  ↳ variables so that if anything is missing there will be no issue
filter(
  !is.na(steps), # ignore n/a for steps
  !is.na(water_intake_l), # ignore n/a for water intake
  !is.na(duration_of_shift_hrs), # ignore n/a for duration of shift
  !is.na(sick_y_n) # ignore n/a for sick
)
```

```
# further editing bubble_data data frame because first draft of graph looked
  ↳ messy
bubble_data <- bubble_data |>
```

```

# using mutate function to
mutate(
  # removes commas from steps column and converts to numerical format
  ↪ rather than character
  steps = as.numeric(gsub(",", "", steps)),
  # creating new variable where steps are /1000 to make x axis cleaner in
  ↪ plot
  steps_k = steps / 1000
)

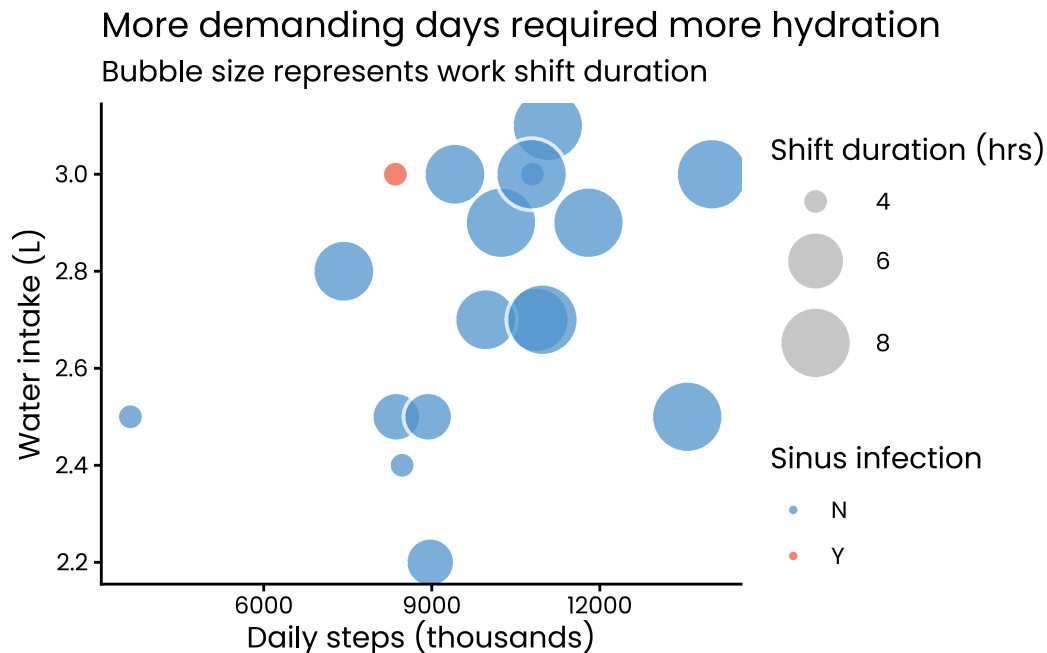
# base layer ggplot with data frame bubble_data
ggplot(data = bubble_data,
  # aes to set axes
  mapping = aes(x = steps, # x axis is daily steps
    y = water_intake_l, # y axis is water intake in liters
    size = duration_of_shift_hrs, # size of bubbles changes by the
    ↪ duration of my shift in hours
    fill = sick_y_n)) + # color is filled by whether or not i am sick
# first layer geom_point to show individual data
geom_point(
  # shape is style 21, which is a filled circle with a colored border, or
  ↪ in this case...
  shape = 21,
  # ...border is white so it looks cool when they overlap!
  color = "white",
  # make the points see through so you can see if there are any overlapping
  alpha = 0.75,
  # controls thickness of border around data points
  stroke = 1
) +
# custom colors
scale_fill_manual(values = c(
  # yes i have a sinus infection = red
  "Y" = "tomato2",
  # no i do not have a sinus infection = blue
  "N" = "steelblue3"
)) +
# customizing bubble size scale
scale_size_continuous(
  # controls minimum and maximum bubble sizes so smaller values result in
  ↪ smaller circles
  range = c(4, 12),

```

```

# specifies which shift duration values should appear in bubble size
  ↪ legend
breaks = c(2, 4, 6, 8)
) +
# setting titles
labs(
  # title states a general idea taken from the plot
  title = "More demanding days required more hydration",
  # subtitle informs reader what bubble size represents more clearly
  subtitle = "Bubble size represents work shift duration",
  # x axis is daily steps measured in thousands of steps
  x = "Daily steps (thousands)",
  # y axis is water intake measured in liters
  y = "Water intake (L)",
  # label to explain that color based on whether or not i had a sinus
  ↪ infection :P
  fill = "Sinus infection",
  # label to explain that size of bubble is based on shift duration with
  ↪ legend
  size = "Shift duration (hrs)"
) +
# nice theme
theme_classic() +
# adding the cutest custom font ever from showtext
theme(
  text = element_text(family = "poppins")
) +
# had to add this in because my legend bubbles kept DISAPPEARING
guides(
  # shape 21 for legend bubbles as well
  size = guide_legend(override.aes = list(shape = 21,
                                           fill = "grey70")) # fill with grey
) +
# using theme to adjust legend because it kept running into subtitle
theme(
  legend.position = "right", # manually making sure legend is all the way to
  ↪ the right
  legend.margin = margin(t = 10) # legend margin t = 10 to give for some
  ↪ breathing room around the legend
)

```



```
# code to save image bubbleplot as a png with specified dimensions
ggsave("visualizations/bubbleplot.png", width = 8, height = 6, dpi = 300)
```

Visualization 3: Ridgeline plot – showing how much water I drink when I have a sinus infection vs do not have a sinus infection

```
# base layer ggplot using my_data_clean data frame
ggplot(data = my_data_clean,
  # setting x and y axes
  mapping = aes(x = water_intake_l, # x axis water intake in liters
    y = sick_y_n, # y axis sick yes or no
    fill = sick_y_n)) + # fill color by sick yes or no
# first layer setting ridgeline densities
geom_density_ridges(
  alpha = 0.7, # making them a little transparent
  color = "white" # creates white border around each ridgeline
) +
# adding custom colors using scale_fill_manual
scale_fill_manual(values = c(
  "Y" = "tomato2", # yes sick is red because this color gives sinus
  ↪ infection
```

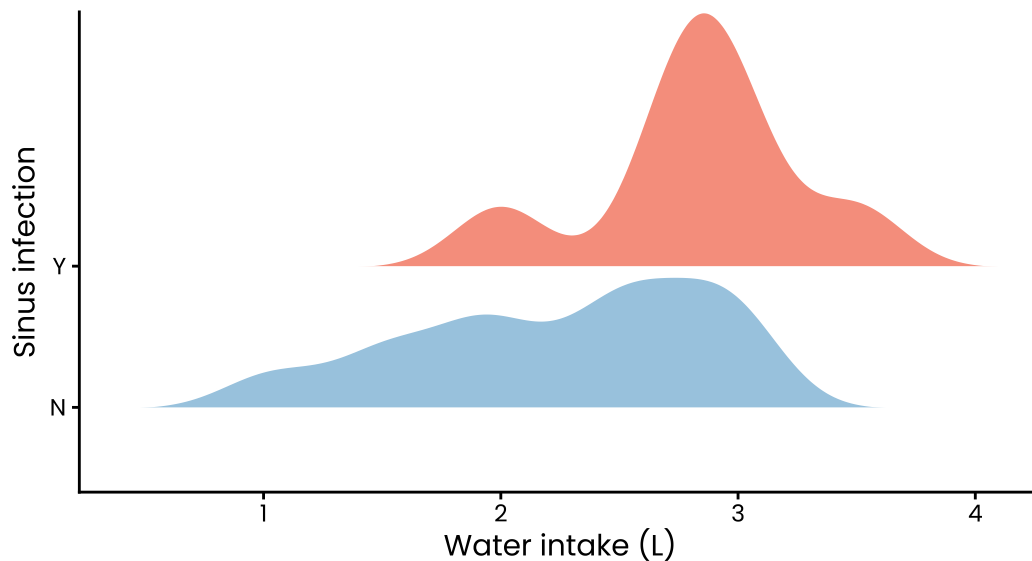
```

  "N" = "skyblue3" # no not sick is blue because this color makes me feel
    ↪ calm, like i do when i do not have a sinus infection
)) +
# labeling title, subtitle, and axes
labs(
  # title is general finding from the figure
  title = "Hydration patterns shifted during illness",
  # subtitle describes the variables being examined in this plot
  subtitle = "Distribution of daily water intake on sick and healthy days",
  # x axis is water intake in liters
  x = "Water intake (L)",
  # y axis is whether or not i have a sinus infection on the day the data
    ↪ is being collected
  y = "Sinus infection"
) +
# setting a nice, classic theme that matches other plots
theme_classic() +
# adding cutest custom font ever from showtext package
theme(
  text = element_text(family = "poppins")
) +
# removing legend, i do not find it to be necessary
theme(legend.position = "none")

```

Hydration patterns shifted during illness

Distribution of daily water intake on sick and healthy days



```
# code to save ridgeline plot as png with specified dimensions
ggsave("visualizations/ridgelineplot.png", width = 8, height = 6, dpi = 300)
```

Bonus: Visualization 4: Pie chart – showing how many days I had a sinus infection for fun

Note: I was honestly shocked at how having a sinus infection greatly affected the amount of water I drink compared to some of the other variables. I think emphasizing this in the infographic would be a fun and personal way to integrate this important variable into my statistical story.

```
# creating new data frame sick_counts from my_data_clean
sick_counts <- my_data_clean |>
  # creating counts of how many observations were taken when i was sick verus
  ↪ not
  count(sick_y_n)
```

```
# editing sick_counts data frame
sick_counts <- sick_counts |>
  # mutating sick_counts data frame to include percentages
  mutate(
```

```

# computing proportions
percent = n / sum(n),
# label for sick
label = ifelse(
  # if row has "Y", it is labeled as being sick
  sick_y_n == "Y",
  # setting handy labels with the percentages to show if i am sick: sick
  ↪ (true condition)
  paste0(scales::percent(percent, accuracy = 1), "\nSick"),
  # setting handy labels with percentages to show if i am sick: not sick
  ↪ (false condition)
  paste0(scales::percent(percent, accuracy = 1), "\nNot sick")
)
)

```

```

# base layer ggplot with sick_counts data frame
ggplot(data = sick_counts,
  # using aes to set axes
  mapping = aes(x = "", # creates single stacked bar for pie chart
    y = n, # using column count for slice sizes in our pie
    fill = sick_y_n)) + # fill color based on whether or not
    ↪ i was sick

# first layer geom_col which makes a bar chart --> pie chart
geom_col(width = 1, # width 1 makes bars full width for pie chart
  color = "white") + # color white adds the white border between the
  ↪ pie slices as seen around the red slice

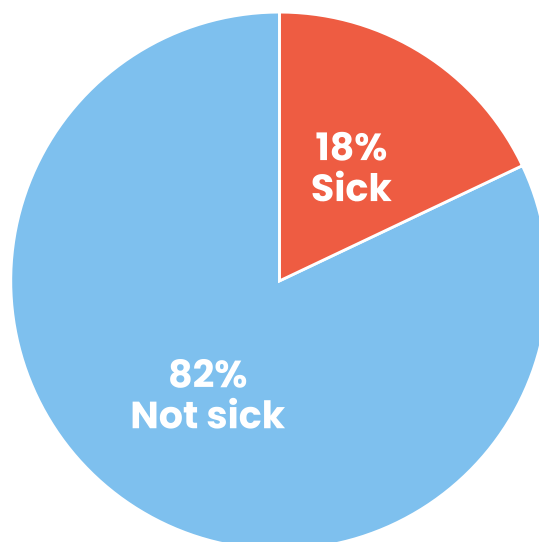
# adding percentage labels directly to pie slices
geom_text(
  aes(label = label),
  position = position_stack(vjust = 0.5), # centers labels within slices
  color = "white", # white text stands out against slice colors
  size = 5,
  lineheight = 0.9,
  family = "poppins",
  fontface = "bold"
) +
# transforms bar chart into circle
coord_polar(theta = "y") + # mapping y values into angles
# custom coloring
scale_fill_manual(

```

```

values = c(
  # yes, sinus infection is red
  "Y" = "tomato2",
  # no, no sinus infection is blue
  "N" = "skyblue2"
),
# setting legend labels
labels = c(
  # "Y" becomes "Yes" so readers can better understand legend
  "Y" = "Yes",
  # "N" becomes "No" so readers can better understand legend
  "N" = "No"
)
) +
# theme void because it gets rid of basically all background elements,
theme_void() +
# adding custom font that matches the other figures!
theme(text = element_text(family = "poppins"),
      # removing legend
      legend.position = "none")

```



```
# code to save pie chart as png with specified dimensions
ggsave("visualizations/piechart.png", width = 8, height = 6, dpi = 300)
```

Write-up: Part 4

General information about design and visualization

- In my jitter plot, I am choosing to highlight how the way that I experience temperature (whether I feel hot, warm, or cold that day) affects the amount of water I drink. I chose this because, during my data collection process, I felt as though the way I experienced temperature made a significant difference in my water intake. As for aesthetic choices, I chose a cute font that meshed well with the rest of my infographic aesthetics, and I decided to add a little label and arrow pointing to the highest average intake because I thought it had a “story-like” component to it that matched by story-telling aspects in my infographic! In addition, I chose intuitive colors for the points (red was hot, yellow was warm, and blue was cold). I feel like utilizing intuitive colors makes it a lot easier to read and interpret.
- In my bubble plot, I am choosing to highlight multiple variables and how they influence the amount of water I drink. The variables considered in this plot include sickness, daily steps, and the duration of my shift. Generally, this plot shows that more demanding days result in a higher water intake. For my visualization, I wanted my data to tell a story about the bigger picture of my day, so I felt as though this was a great way to do that. As for my aesthetic choices, I used a consistent font throughout my plots, and I decided to make my non-sick data bubbles blue and my sick data bubble red. I liked this because it makes the red dot stick out more, which is good, because it had an unusually high water intake for the duration of shift. The fact that I was sick explains why.
- In my ridge line plot, I am choosing to highlight how being sick influences how much water I drink. I decided use this variable for this plot because I was genuinely shocked at how much being sick affected my water intake! As for my aesthetic choices, I chose to stay consistent with the font once again, and use red and blue again, just as I had for the bubble plot. I felt as though this consistency, again, makes my plots easier to read. I think that when you are reading an infographic and begin to correlate certain colors with certain variables, changing these in future plots just causes confusion for the reader. In general, this plot was heavily an aesthetic decision, as I wanted to have a fun plot with shapes that had more visual interest! I wanted my figures to have variety, just as my variables and daily intake did, and this is a consequence of that goal. It makes my infographic a little more fun to look at, and my narrative a little more interesting.
- I decided to create an additional pie chart to show the percentage of how much I was sick. This is because I was sick far more than usual during my data collection process,

which made the results different from how they would be typically. I felt as though this pie chart was a great compliment to the ridge line plot demonstrating how the amount of water that I drink shifts when I'm sick. As for my aesthetic choices, I chose to bold the letters to make it stick out more to the reader, and I made sure that the little sick "pie slice" had a border so it was easier to see. Again, I utilized the same colors that I have been associating with being sick or not sick in order to avoid confusing the reader. This was mostly a supporting figure for the ridge line plot, but the figure itself adds to the narrative of my story because it shows how I was uncharacteristically sick for almost 1/5 of my data collection. I think this also enhances the work because it adds another variable that everyone can relate to.

Sources and processes

Inspirations

- My jitter plot was inspired by Cedric Scherer's "Bill dimensions of brush-tailed penguins" scatter plot. The way that he chose to differently color each type of penguin led me to choose to color each temperature bucket in a different way.
- My bubble plot was actually inspired by a blog post that I found on the Minitab blog. This particular post by Eric Heckman was titled, "Introducing the Bubble Plot," and I instantly loved the way it looked. I knew I wanted my infographic to have a lot of fun, personal components, and I found bubble plots to look whimsical and fun.
- My ridge line plot was heavily inspired by Sam Csik's "One workflow for building effective (and pretty) {ggplot2} data visualizations," as they have an incredible example. I utilized this in a simpler way, but I simply loved the visual interest the shapes provided.
- Finally, my pie chart was actually inspired by Deepanshu Bhalla's "CREATE INFO-GRAPHICS WITH R" images and icons site. This is because I wanted my pie chart to appear more as a big, informative icon, which is also why I put a fun border around it and made all of the necessary information available on the pie chart itself!

Data creation process

- For my jitter plot, I did have to clean all of my data. I cleaned the names using the janitor package (made my columns lowercase and include underscores), and I also created and labeled temperature levels (hot, warm, and cold) so that my levels would be in the correct order with the correct labels attached to them! I also made sure to remove any columns with no temperature data just in case. While coding this figure, the hardest part was definitely adding that arrow and positioning it correctly because the label kept running into everything!

- For my bubble plot, I used my already cleaned data from my jitter plot, but I also made sure to make it so that any columns with no variable data were removed. I accidentally added an empty row in my excel where I recorded my data (oops) so I had to take care of that because Rstudio (rightfully) kept getting confused. Then, my first draft looked super messy because my steps data were all over the place, so I also added code to make it so that step counts were in specific increments.
- For my ridge line plot, I used my cleaned data from my jitter plot. This plot actually gave me the least amount of problems while trying to construct it, which I was thankful for after having to tweak my bubble lot *a lot*. I enjoyed making this, it was definitely one of my favorite ones to make.
- For my pie chart, I created a new data frame of counts of each time I was sick versus not sick to begin with. Then, I created percentages from these counts and also put in code to create labels associated with these percentages: sick and not sick. From there, I was able to code my figure successfully!

Tools utilized

I utilized a lot of the resources provided to me on the sites where I found inspirations for these figures (as shared above)! These shared the right geoms and functions for forming these new visualizations. In addition, I did utilize ChatGPT to troubleshoot my code and understand why certain things may not have been working correctly when I expected them to.